TP2 - Failles Web & Système d'exploitation

Mathieu WAHARTE - APP5

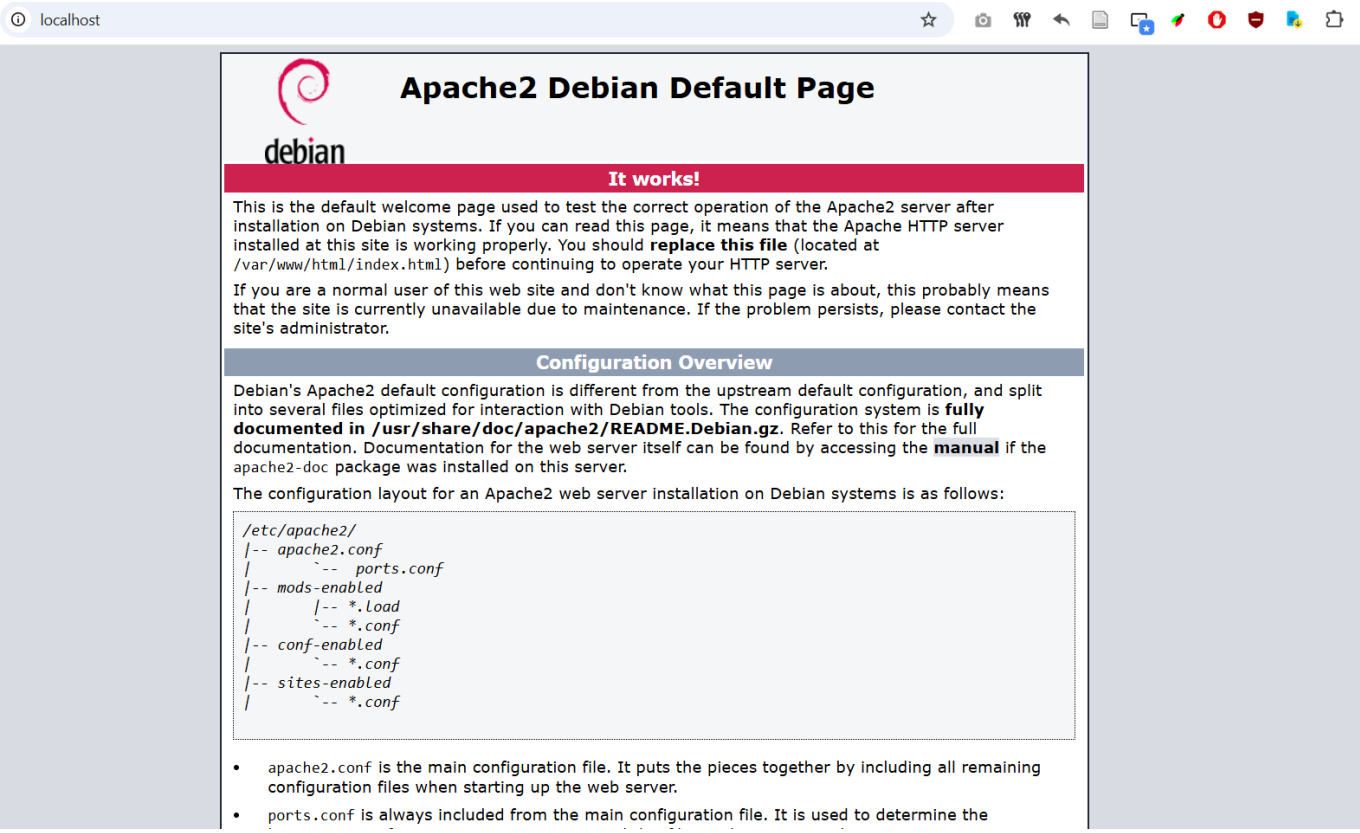
Exercice 1 - XSS

1. On observe avec `lsof -i` que Apache2 est lancé:

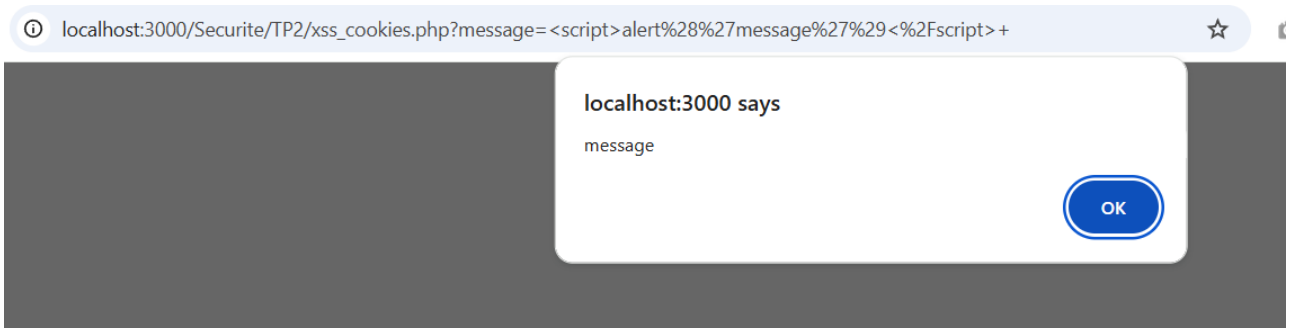
```
root@LAPTOP-QJHCANDR:~# lsof -i
COMMAND   PID     USER   FD   TYPE DEVICE SIZE/OFF NODE NAME
sshd      138     root    3u  IPv4   2283      0t0  TCP *:ssh (LISTEN)
sshd      138     root    4u  IPv6   2285      0t0  TCP *:ssh (LISTEN)
node      519  bluefir   22u  IPv4   3200      0t0  TCP localhost:46031 (LISTEN)
node      519  bluefir   26u  IPv4  66515      0t0  TCP localhost:46031->localhost:54528 (ESTABLISHED)
node      519  bluefir   36u  IPv4  36286      0t0  TCP 172.27.248.70:44286->150.171.73.16:https (ESTABLISHED)
node      519  bluefir   37u  IPv4  36287      0t0  TCP 172.27.248.70:44302->150.171.73.16:https (ESTABLISHED)
node      519  bluefir   39u  IPv4  76505      0t0  TCP 172.27.248.70:53104->150.171.74.16:https (ESTABLISHED)
node      519  bluefir   40u  IPv4  36290      0t0  TCP 172.27.248.70:53108->150.171.74.16:https (ESTABLISHED)
node      519  bluefir   41u  IPv4  36289      0t0  TCP 172.27.248.70:53106->150.171.74.16:https (ESTABLISHED)
node      519  bluefir   42u  IPv4  76506      0t0  TCP 172.27.248.70:44312->150.171.73.16:https (ESTABLISHED)
node      519  bluefir   43u  IPv4  36288      0t0  TCP 172.27.248.70:44316->150.171.73.16:https (ESTABLISHED)
node      519  bluefir   45u  IPv4  36291      0t0  TCP 172.27.248.70:53120->150.171.74.16:https (ESTABLISHED)
node     5462  bluefir   22u  IPv4  79941      0t0  TCP localhost:54528->localhost:46031 (ESTABLISHED)
node     5471  bluefir   22u  IPv4  79954      0t0  TCP localhost:54538->localhost:46031 (ESTABLISHED)
node     5478  bluefir   22u  IPv4   3761      0t0  TCP localhost:46031->localhost:54538 (ESTABLISHED)
node     5478  bluefir   67u  IPv6  76510      0t0  TCP *:58045 (LISTEN)
java     6127  bluefir   45u  IPv6  71847      0t0  TCP localhost:40763 (LISTEN)
java     6127  bluefir   48u  IPv6  74695      0t0  TCP localhost:64120 (LISTEN)
apache2 17164     root    4u  IPv6 178677      0t0  TCP *:http (LISTEN)
apache2 17167 www-data    4u  IPv6 178677      0t0  TCP *:http (LISTEN)
apache2 17168 www-data    4u  IPv6 178677      0t0  TCP *:http (LISTEN)
apache2 17169 www-data    4u  IPv6 178677      0t0  TCP *:http (LISTEN)
apache2 17170 www-data    4u  IPv6 178677      0t0  TCP *:http (LISTEN)
apache2 17171 www-data    4u  IPv6 178677      0t0  TCP *:http (LISTEN)
```

   Le processus principal est `apache2` et il est lancé par l'utilisateur `root`. Il lance ensuite des processus fils `apache2` qui sont lancés par l'utilisateur `www-data` (le serveur web).

Le serveur apache est bien accessible à localhost.

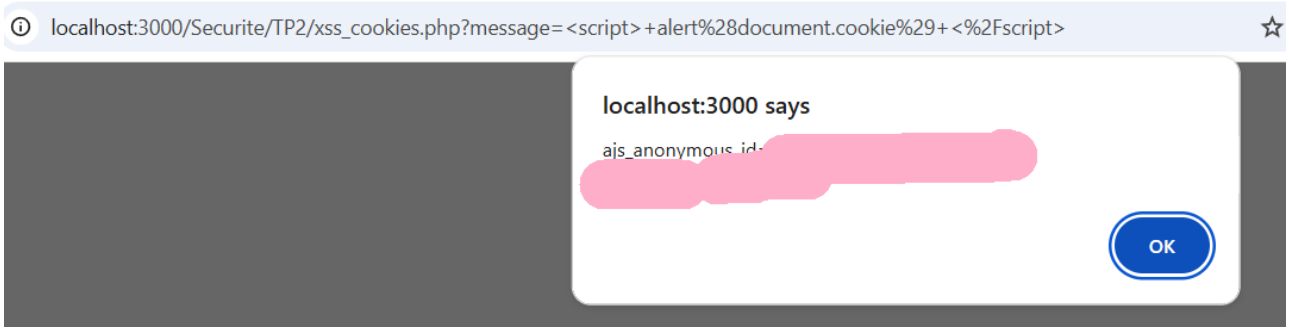


2. J'ai servi la page `xss_cookies.php` et j'ai entré `<script> alert('message') </script>` dans le champ de saisie:



On observe que le script est exécuté et affiche une alerte avec le message "message". Cela montre que la page est vulnérable à une attaque de type Cross-Site Scripting (XSS), car elle ne filtre pas correctement les entrées utilisateur avant de les afficher, on les écrit sans filtrage ou validation dans un fichier qu'on lit ensuite directement, toujours sans vérifications. Comme proposé au début, on pourrait envoyer les cookies utilisateurs (logins bancaires, comptes de réseaux sociaux, etc.) à un serveur malveillant pour les exploiter ensuite.

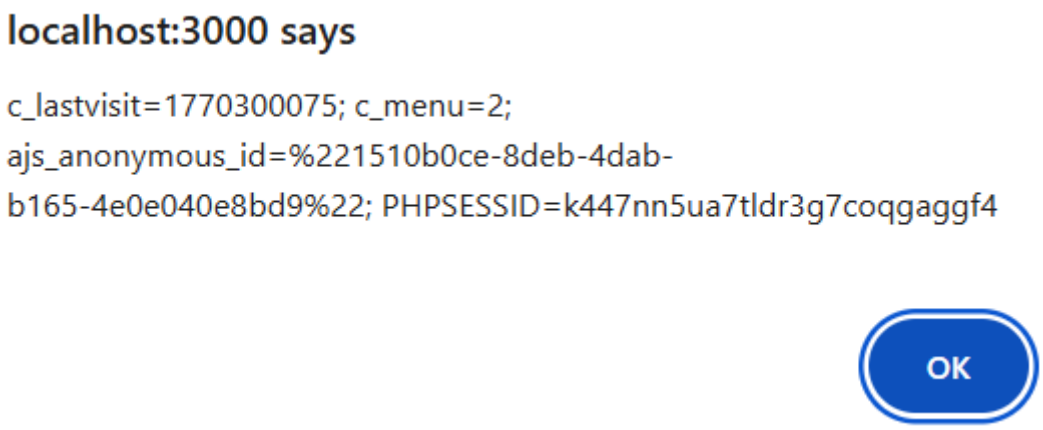
3. En remplaçant le script par `<script> alert(document.cookie) </script>`, on observe maintenant les cookies de mon navigateur:



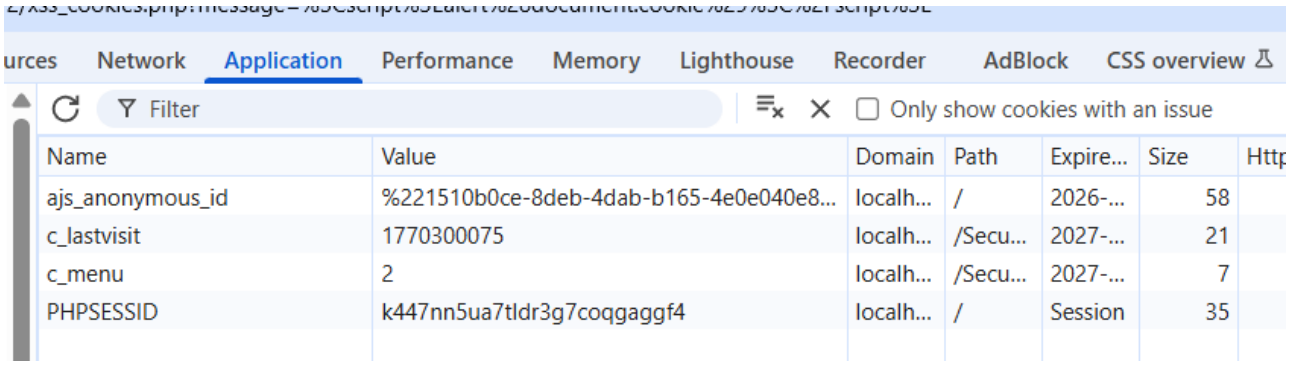
Cela montre que les cookies sont accessibles via le script injecté, ce qui peut être dangereux si les cookies contiennent des informations sensibles. Un attaquant pourrait utiliser cette vulnérabilité pour voler les cookies de session d'un utilisateur et ainsi prendre le contrôle de sa session sur le site web. On pourrait aussi profiter d'une vulnérabilité dans le moteur de rendu du navigateur (V8 pour Chrome) et ainsi exécuter du code directement sur la machine de l'utilisateur, ce qui est encore plus dangereux.

4. Le navigateur conserve les cookies même si on termine le navigateur car les cookies `c_lastvisit` et `c_menu` ont une date d'expiration définie dans le futur (dans session.php). En revanche la session est perdue si on termine le navigateur car les cookies de session n'ont pas de date d'expiration et sont supprimés lorsque le navigateur est fermé.

5. Pour afficher les cookies, on peut tout simplement utiliser `<script>alert(document.cookie)</script>` comme on l'a fait précédemment. On obtient alors:

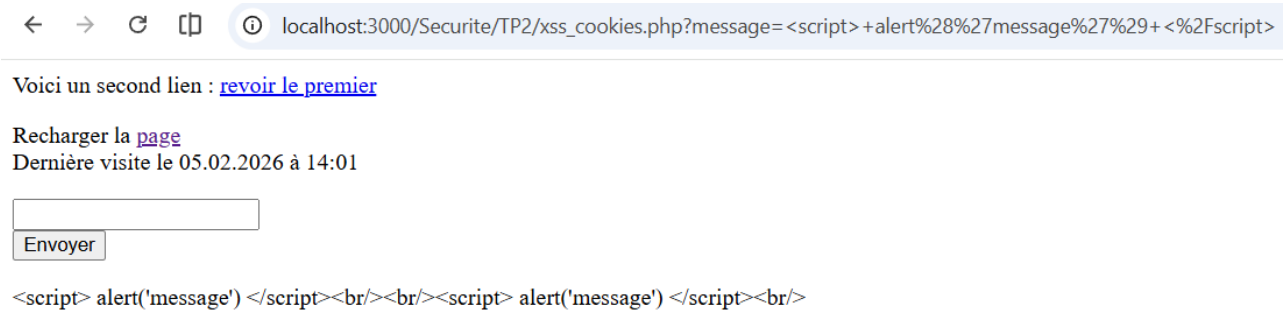


On trouve bien les cookies `c_lastvisit` et `c_menu` qui sont persistants, et le cookie de session `PHPSESSID` qui est supprimé lorsque le navigateur est fermé. On peut vérifier cela dans les Chrome DevTools:



6. La valeur du cookie pour la dernière visite est de `1770300075` qui est un temps en Unix mili (temps écoulé depuis le 1er Janvier 1970 en secondes). En convertissant ce temps en date, on trouve que la dernière visite a eu lieu le 5 Février 2026 à 14:01:15 UTC.

7. En mettant `htmlentities()` autours de l'entrée utilisateur, on convertit les caractères spéciaux en entités HTML, ce qui empêche l'exécution de scripts malveillants. Par exemple, `<script>` devient `<script>`, ce qui est affiché comme du texte normal dans le navigateur au lieu d'être interprété comme du code. Cela protège contre les attaques XSS en empêchant l'injection de code malveillant dans la page web. Cette solution n'est cependant pas suffisante pour protéger contre toutes les attaques XSS, car il existe d'autres vecteurs d'attaque (comme les événements JavaScript ou les URL malveillantes) qui peuvent encore être exploités. De plus cela peut rendre l'expérience utilisateur moins agréable si les utilisateurs veulent utiliser des caractères spéciaux dans leurs entrées. J'ai été confronté à ce cas au travail, je devais en même temps protéger contre ces attaques dans un champ texte mais il était spécifié originellement que les utilisateurs pouvaient utiliser `<` et `>` dans leurs entrées. Comme la taille du champs était aussi contrainte, j'ai discuté avec mes collègues et on a choisi d'enlever ces caractères du champs.
   Si on reassaye notre injection `<script> alert('message') </script>` après avoir ajouté `htmlentities()`, on observe que le script n'est plus exécuté et est affiché comme du texte normal:



8. Dans `messages.txt`, on trouve bien les entités HTML au lieu des caractères spéciaux:

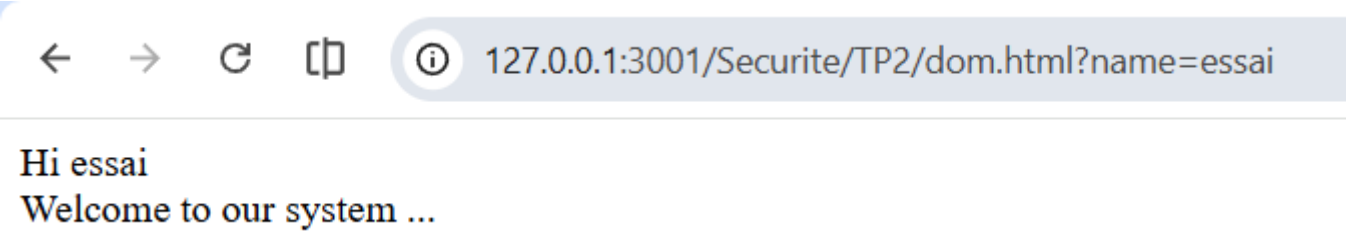
```
You, 30 seconds ago | 1 author (You)
&lt;script&gt; alert(&#039;message&#039;) &lt;/script&gt;&lt;br/&gt;&lt;br/&gt;&lt;
```

   Ce type de "sanization" permet aussi d'éviter de corrompre sa propre base de données ou son application.

9. Je conclu que les failles XSS sont très dangereuses car elles permettent à un attaquant d'injecter du code malveillant dans une page web, ce qui peut entraîner le vol de données sensibles, la prise de contrôle de la session d'un utilisateur, ou même l'exécution de code sur la machine de l'utilisateur. Heureusement, il existe des moyens de s'en prémunir, il faut déjà nettoyer **toutes** les entrées utilisateur (dont les API), de plus il faut les valider à chaque niveau de l'application (le back-end ne peut pas faire confiance ni au front-end ni à la base de données). Il faut aussi éviter d'afficher des données sensibles dans les cookies ou les URL, et utiliser des mécanismes de sécurité comme les Content Security Policy (CSP) pour limiter les sources de scripts autorisées. Enfin, je dirais que malgré leur risques, les failles XSS sont assez évitables, notamment par l'utilisation d'outils à jour et d'analyseurs de code statique (ce type de vulnérabilité peut être détecté automatiquement).

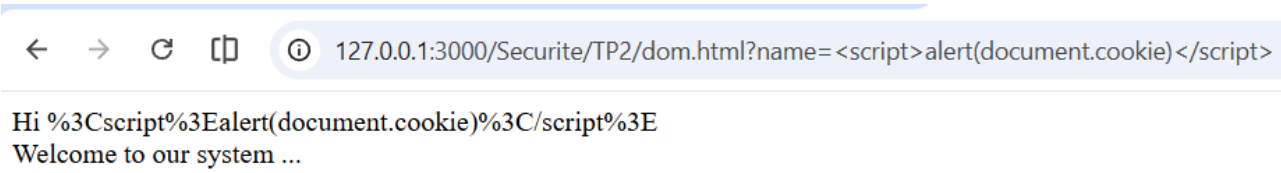
Exercice 2 - DOM

Avec essai dans l'URL, `dom.html` affiche bien le bon message:



10. On change l'URL en `dom.html?name=<script>alert(document.cookie)</script>` pour essayer d'injecter du code JavaScript.

11. L'affichage obtenu est le suivant:

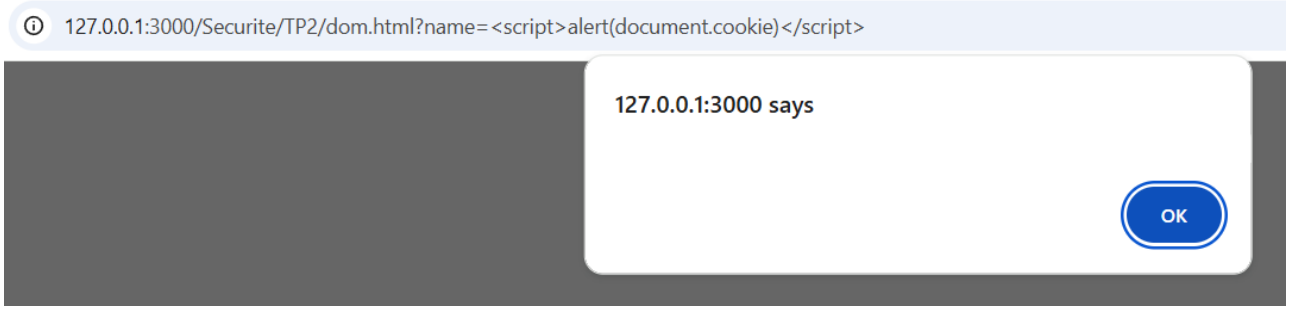
    

    Le script n'est pas exécuté, il est affiché comme un texte mais les caractères spéciaux sont échappés. Le script par défaut n'interprète pas les caractères spéciaux, il les affiche tels quels. Ce type d'injection XSS n'est pas possible par défaut.

12. Pour faire fonctionner la faille, on peut utiliser la fonction `decodeURIComponent()` pour décoder les caractères spéciaux dans l'URL. En modifiant le script dans `dom.html` comme suit:

    ```
    <html>
      <title>Welcome!</title>
      Hi
      <script>
        const pos = document.URL.indexOf("name=") + 5;
        document.write(decodeURIComponent(document.URL.substring(pos,
    document.URL.length)));
      </script>
      <br />
      Welcome to our system ...
    </html>
    ```

    On peut alors injecter du code JavaScript avec un paramètre dans l'URL comme `dom.html?name=<script>alert(document.cookie)</script>`, on a alors:



On ne voit pas de cookies car il n'y en a pas dans ce contexte mais le script est exécuté, on vient donc d'ouvrir la porte aux failles XSS en utilisant `decodeURIComponent()` pour décoder les caractères spéciaux dans l'URL. Cela montre que même si le script par défaut n'interprète pas les caractères spéciaux, il est possible de contourner cette protection en utilisant des fonctions de décodage, ce qui souligne l'importance de valider et de nettoyer toutes les entrées utilisateur, y compris celles provenant de l'URL.

Exercice 3 - Comptes utilisateurs et mots de passe

On va créer un premier utilisateur avec la commande `useradd` et lui donner un mot de passe avec la commande `passwd`:

```
root@LAPTOP-QJHCANDR:~# useradd mock1
root@LAPTOP-QJHCANDR:~# passwd mock1 pass
Usage: passwd [options] [LOGIN]

Options:
  -a, --all                    report password status on all accounts
  -d, --delete                 delete the password for the named account
  -e, --expire                 force expire the password for the named account
  -h, --help                   display this help message and exit
  -k, --keep-tokens            change password only if expired
  -i, --inactive INACTIVE      set password inactive after expiration
                               to INACTIVE
  -l, --lock                   lock the password of the named account
  -n, --mindays MIN_DAYS       set minimum number of days before password
                               change to MIN_DAYS
  -q, --quiet                  quiet mode
  -r, --repository REPOSITORY  change password in REPOSITORY repository
  -R, --root CHROOT_DIR        directory to chroot into
  -S, --status                 report password status on the named account
  -u, --unlock                 unlock the password of the named account
  -w, --warndays WARN_DAYS     set expiration warning days to WARN_DAYS
  -x, --maxdays MAX_DAYS       set maximum number of days before password
                               change to MAX_DAYS

root@LAPTOP-QJHCANDR:~# passwd mock1
New password:
Retype new password:
passwd: password updated successfully
```

Le fichier contenant les données système du compte est `/etc/passwd`:

`mock1:x:1001:1002::/home/mock1:/bin/sh`

Et le fichier contenant l'empreinte du mot de passe est `/etc/shadow`:
`mock1:$y$j9T$XjqWXlFKO0Ad9bkhNqd3n0$PVJPFytpbEDhBkn28XVPPtttVI3xuFJhmKUvixhCaR5:20489:0:99999:7:::`

Voici le format d'empreinte d'après la page de manuel de `crypt`:
`$ID$SALT$HASH`

- `ID` est l'identifiant de la fonction de hash utilisée (par exemple, `1` pour MD5, `5` pour SHA-256, `6` pour SHA-512, etc.)
- `SALT` est une chaîne de caractères aléatoire utilisée pour rendre le hash plus résistant aux attaques par dictionnaire et par force brute.
- `HASH` est le résultat du hashage du mot de passe combiné avec le salt.

```
FORMAT OF HASHED PASSPHRASES
     All of the hashing methods supported by crypt(3) produce a hashed passphrase which consists of four components:
     prefix, options, salt, and hash.  The prefix controls which hashing method is to be used, and is the appropriate
     string to pass to crypt_gensalt(3) to select that method.  The contents of options, salt, and hash are up to the
     method.  Depending on the method, the prefix and options components may be empty.

     The setting argument to crypt(3) must begin with the first three components of a valid hashed passphrase, but
     anything after that is ignored.  This makes authentication simple: hash the input passphrase using the stored
     passphrase as the setting, and then compare the result to the stored passphrase.

     Hashed passphrases are always entirely printable ASCII, and do not contain any whitespace or the characters ':',
     ';', '*', '!', or '\'.  (These characters are used as delimiters and special markers in the passwd(5) and
     shadow(5) files.)

     The syntax of each component of a hashed passphrase is up to the hashing method.  '$' characters usually delimit
     components, and the salt and hash are usually encoded as numerals in base 64.  The details of this base-64 en-
     coding vary among hashing methods.  The common "base64" encoding specified by RFC 4648 is usually not used.
```

13. En regardant l'empreinte du mot de passe dans `/etc/shadow`, on observe que la fonction de hash utilisée est yescrypt, car l'identifiant `ID` est égal à `y`.

```
yescrypt
  yescrypt is a scalable passphrase hashing scheme designed by Solar Designer, which is based on Colin Percival's
  scrypt.  Recommended for new hashes.

  Prefix
      "$y$"

  Hashed passphrase format
      \$y\$[./A-Za-z0-9]+\$[./A-Za-z0-9]{,86}\$[./A-Za-z0-9]{43}

  Maximum passphrase length
      unlimited

  Hash size
      256 bits

  Salt size
      up to 512 (128+ recommended) bits

  CPU time cost parameter
      1 to 11 (logarithmic)
```

La ligne complète (trouvée précédemment) est :
`mock1:$y$j9T$XjqWXlFKO0Ad9bkhNqd3n0$PVJPFytpbEDhBkn28XVPPtttVI3xuFJhmKUvixhCaR5:20489:0:99999:7:::`

On crée un nouvel utilisateur `mock2` et on lui donne un mot de passe avec `openssl passwd -1` (`-crypt` n'est pas disponible dans ma version d'openssl) qu'on va mettre directement dans le fichier `/etc/shadow` pour ce nouvel utilisateur. L'empreinte du mot de passe de `mock2` est la suivante:
`$1$LW3HcSWk$ISBdqLWym/VEKAqlyAOG51`

```
root@LAPTOP-QJHCANDR:~# openssl passwd -1
Password:
Verifying - Password:
$1$LW3HcSWk$ISBdqLWym/VEKAqlyAOG51
```

```
mock2:$1$LW3HcSWk$ISBdqLWym/VEKAqlyAOG51:20489:0:99999:7:::
```

Ce mot de passe fonctionne bien (avec la commande `su - mock2`):

```
bluefir  ~  su - mock2
Password:
su: warning: cannot change dire
$
```

Bien que le hachage MD5 soit différent du DES demandé (non disponible), le principe de sécurité démontré (différence de hash due au sel) reste identique.

14. On va comparer les deux empreintes des mots de passe des deux utilisateurs créés.
    L'empreinte du mot de passe de `mock1` est la suivante:
    `$y$j9T$XjqWXlFKO0Ad9bkhNqd3n0$PVJPFytpbEDhBkn28XVPPtttVI3xuFJhmKUvixhCaR5`.
    Et l'empreinte du mot de passe de `mock2` est la suivante:
    `$1$LW3HcSWk$ISBdqLWym/VEKAqlyAOG51`.
    Les deux chaînes sont différentes car elles utilisent des fonctions de hash différentes (yescrypt pour `mock1` et MD5 pour `mock2`), et elles ont aussi des salts différents (`XjqWXlFKO0Ad9bkhNqd3n0` pour `mock1` et `LW3HcSWk` pour `mock2`). Et ce pour le même mot de passe, ce qui montre l'importance du salt pour rendre les hash plus résistants aux attaques par dictionnaire et par force brute. En effet, même si deux utilisateurs ont le même mot de passe, leurs empreintes seront différentes grâce au salt, ce qui rend plus difficile pour un attaquant de deviner les mots de passe à partir des empreintes. De plus, l'utilisation de fonctions de hash plus robustes comme yescrypt rend également les empreintes plus résistantes aux attaques.

Pour tester John, on va mettre le mot de passe `j1l2t3` qu'on retrouve dans son dictionnaire:

```
root@LAPTOP-QJHCANDR:~# tail -40 /usr/share/john/password.lst
i
ilmari
irmeli
j1l2t3
```

15. En lançant avec la commande `#john --wordlist=/usr/share/john/password.lst –format=crypt /etc/shadow`, on observe que le mot de passe est retrouvé en 59 secondes avec 16 threads:

```
root@LAPTOP-QJHCANDR:~# john --wordlist=/usr/share/john/password.lst --format=crypt /etc/shadow
Loaded 3 password hashes with 3 different salts (crypt, generic crypt(3) [?/64])
Will run 16 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
0g 0:00:00:24 45% 0g/s 57.76p/s 173.2c/s 173.2C/s grumpy..keeper
0g 0:00:00:34 61% 0g/s 57.78p/s 173.3c/s 173.3C/s pretty..celtic
0g 0:00:00:45 78% 0g/s 56.61p/s 174.0c/s 174.0C/s 10sne1..nermal
pass             (mock1)
j1l2t3           (mock2)
2g 0:00:00:59 100% 0.03389g/s 60.10p/s 173.8c/s 173.8C/s !@#$%..sss
Use the "--show" option to display all of the cracked passwords reliably
Session completed
```

Pour les mots de passe dans le dictionnaire, John est très efficace pour les retrouver, même avec des fonctions de hash plus robustes comme yescrypt.

Maintenant on mets le mot de passe `wx` qui n'est pas dans le dictionnaire de John (`grep wx /usr/share/john/password.lst` ne retourne rien):

```
root@LAPTOP-QJHCANDR:~# grep wx /usr/share/john/password.lst
root@LAPTOP-QJHCANDR:~# grep j1l2t3 /usr/share/john/password.lst
j1l2t3
```

16. En relançant la commande de recherche, on observe que le mot de passe n'est pas retrouvé (mais l'autre toujours), même après un certain temps d'attente:

```
root@LAPTOP-QJHCANDR:~# john --wordlist=/usr/share/john/password.lst --format=crypt /etc/shadow
Loaded 3 password hashes with 3 different salts (crypt, generic crypt(3) [?/64])
Will run 16 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
0g 0:00:00:22 61% 0g/s 91.30p/s 273.9c/s 273.9C/s pretty..celtic
0g 0:00:00:29 78% 0g/s 89.07p/s 273.8c/s 273.8C/s 10sne1..nermal
pass             (mock1)
1g 0:00:00:37 100% 0.02656g/s 94.18p/s 272.5c/s 272.5C/s !@#$%..sss
Use the "--show" option to display all of the cracked passwords reliably
Session completed
```

Cela montre que les mots de passe qui ne sont pas dans les dictionnaires peuvent être plus difficiles à retrouver pour les attaquants, surtout s'ils sont courts et simples. Cependant, il est important de noter que les attaques par force brute peuvent toujours être utilisées pour essayer de deviner les mots de passe, même s'ils ne sont pas dans les dictionnaires, ce qui souligne l'importance d'utiliser des mots de passe longs et complexes pour renforcer la sécurité des comptes utilisateurs.

On mets à jour la configuration de john avec un nouveau mode de recherche sur 2 caractères (en corrigeant les typos):

```
CharCount = 10

[Incremental:essai]
File = /usr/share/john/ascii.chr
MinLen=2
MaxLen=2
|
# Some pre-defined word filters a
```

17. On lance La recherche avec la configuration pour 2 caractères avec `john --format=crypt -incremental=essai /etc/shadow` (typo dans l'énoncé):

```
root@LAPTOP-QJHCANDR:~# john --format=crypt --incremental=essai /etc/shadow
Loaded 3 password hashes with 3 different salts (crypt, generic crypt(3) [?/64])
Remaining 2 password hashes with 2 different salts
Will run 16 OpenMP threads
Warning: only 95 characters available
Press 'q' or Ctrl-C to abort, almost any other key for status
wd               (mock2)
1g 0:00:01:01 0.01628g/s 146.9p/s 162.5c/s 162.5C/s ||
Use the "--show" option to display all of the cracked passwords reliably
Session completed
```

On observe que `wd` est maintenant trouvé en 1 minute. Le temps de recherche est plus long que pour les mots de passe dans le dictionnaire, voyons ce que cela donne pour 4 caractères.

En mettant à nouveau à jour la configuration de john pour 4 caractères (remplacer 2 par 4 dans la configuration de essai), on relance la recherche:

```
root@LAPTOP-QJHCANDR:~# john --format=crypt --incremental=essai /etc/shadow
Loaded 3 password hashes with 3 different salts (crypt, generic crypt(3) [?/64])
Remaining 1 password hash
Will run 16 OpenMP threads
Warning: only 95 characters available
Press 'q' or Ctrl-C to abort, almost any other key for status
0g 0:00:53:13 0g/s 171.0p/s 171.0c/s 171.0C/s oozt..oo86
0g 0:00:53:20 0g/s 171.0p/s 171.0c/s 171.0C/s oshr..osck
0g 0:01:06:17 0g/s 171.4p/s 171.4c/s 171.4C/s og4n..ov7p
0g 0:01:10:12 0g/s 171.0p/s 171.0c/s 171.0C/s rvcu..rvgm
0g 0:01:34:38 0g/s 169.8p/s 169.8c/s 169.8C/s o0ya..o0pD
0g 0:01:38:27 0g/s 169.6p/s 169.6c/s 169.6C/s LI92..LIJd
0g 0:01:43:04 0g/s 169.1p/s 169.1c/s 169.1C/s 34eq..380c
0g 0:01:45:52 0g/s 168.8p/s 168.8c/s 168.8C/s t-14..t-o0
0g 0:01:46:59 0g/s 168.9p/s 168.9c/s 168.9C/s 4m21..4mg2
0g 0:01:49:56 0g/s 169.6p/s 169.6c/s 169.6C/s v1og..v1r3
0g 0:01:51:14 0g/s 169.8p/s 169.8c/s 169.8C/s 1tpf..1t5o
0g 0:01:52:15 0g/s 170.0p/s 170.0c/s 170.0C/s t.nv..t.w5
0g 0:01:56:33 0g/s 170.9p/s 170.9c/s 170.9C/s 3OK6..3OTk
0g 0:02:01:44 0g/s 174.2p/s 174.2c/s 174.2C/s nxl;..nxsn
```

J'ai stoppé la recherche après 2 heures d'attente, elle n'avait pas encore trouvé le mot de passe.
La configuration est bien:

```
[Incremental:essai]
File = /usr/share/john/ascii.chr
MinLen=4
MaxLen=4
```

Ainsi, on observe que la recherche prend beaucoup plus de temps pour un mot de passe de 4 caractères, et il est possible que le mot de passe ne soit pas retrouvé du tout dans un temps raisonnable (croissance exponentielle). Cela montre que les mots de passe plus longs sont généralement plus difficiles à deviner pour les attaquants, même avec des fonctions de hash robustes, ce qui souligne l'importance d'utiliser des mots de passe longs et complexes pour renforcer la sécurité des comptes utilisateurs.

Exercice 4 - Comptes et droits Unix

Vérifions que lorsqu'on change le mot de passe d'un utilisateur non privilégié, le fichier `/etc/shadow` est bien modifié:

```
root@LAPTOP-QJHCANDR:~# tail -4 /etc/shadow
messagebus:!:20075::::::
sshd:!:20334::::::
mock1:$y$j9T$vRWWGIDyWkr7oOC/YrRwC1$vMK/VKDlFbrdjEmWlgacgq36.EBSYax.nUnHKSuxrU.:20489:0:99999:7:::
mock2:$y$j9T$9RRlwSK0Uz1FKCxFZTVHD0$79ZlhMOtO38aoyrnohFlqKutB7sAAgodxdZMITSx9z6:20489:0:99999:7:::
root@LAPTOP-QJHCANDR:~# passwd mock2
New password:
Retype new password:
passwd: password updated successfully
root@LAPTOP-QJHCANDR:~# tail -4 /etc/shadow
messagebus:!:20075::::::
sshd:!:20334::::::
mock1:$y$j9T$vRWWGIDyWkr7oOC/YrRwC1$vMK/VKDlFbrdjEmWlgacgq36.EBSYax.nUnHKSuxrU.:20489:0:99999:7:::
mock2:$y$j9T$3mvQDAdLQ9tbGsu9TWPJx0$chs6UGP4PkTKbm6HvXeYmbsnEylLaC1.14WTHYVaJe4:20490:0:99999:7:::
```

18. Les droits pour le fichier `/etc/shadow` sont `-rw-r-----` avec le propriétaire `root` et le groupe `shadow`. Cela signifie que seul l'utilisateur root a les droits de lecture et d'écriture sur ce fichier, tandis que les membres du groupe shadow ont uniquement les droits de lecture, et les autres utilisateurs n'ont aucun droit.

```
root@LAPTOP-QJHCANDR:~# ls -l /etc/shadow
-rw-r----- 1 root shadow 848 Feb  6 14:02 /etc/shadow
```

Les droits pour le programme `passwd` sont `-rwsr-xr-x` avec le propriétaire `root` et le groupe `root`. Cela signifie que le programme `passwd` a les droits de lecture, d'écriture et d'exécution pour le propriétaire root, les droits de lecture et d'exécution pour les membres du groupe root, et les droits de lecture et d'exécution pour les autres utilisateurs.

```
root@LAPTOP-QJHCANDR:~# ls -l /usr/bin/passwd
-rwsr-xr-x 1 root root 68248 Dec 14 15:00 /usr/bin/passwd
```

De plus, le bit setuid est activé (indiqué par le `s` dans les droits), ce qui signifie que lorsque le programme `passwd` est exécuté par un utilisateur non privilégié, il s'exécute avec les privilèges de l'utilisateur root. C'est pourquoi le fichier `/etc/shadow` a été modifié par mon compte non privilégié lorsque j'ai changé le mot de passe depuis `mock2`, car le programme `passwd` a les droits nécessaires pour modifier ce fichier grâce au bit setuid, même si l'utilisateur qui exécute le programme n'a pas les droits directs sur le fichier `/etc/shadow`. On peut trouver ces détails dans la page de manuel de `chmod`:

```
The letters rwxXst select file mode bits for the affected users: read (r), write (w), execute
directories)  (x),  execute/search  only if the file is a directory or already has execute per
user (X), set user or group ID on execution (s), restricted deletion flag or sticky bit (t).
or  more of these letters, you can specify exactly one of the letters ugo: the permissions gra
```

On écrit et compile le programme `shell_prop.c` sous root:

```
root@LAPTOP-QJHCANDR:~# cat /tmp/shell_prop.c
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
void main(void)
{
/* pour forcer l'uid reel effectif et sauvé à root */
setreuid(0,0);
/* lancer le shell */
execl("/bin/bash","bash",0,NULL);
}
root@LAPTOP-QJHCANDR:~# gcc /tmp/shell_prop.c -o /tmp/shell_prop
```

Ensuite on défini le propriété comme root avec `chown root:root /tmp/shell_prop`:

```
root@LAPTOP-QJHCANDR:~# chown root:root /tmp/shell_prop
root@LAPTOP-QJHCANDR:~#
```

On ajoute ensuite le bit setuid et les droits d'exécution pour les utilisateurs avec `chmod u+s,a+x /tmp/shell_prop`:

```
root@LAPTOP-QJHCANDR:~# chmod u+s,a+x /tmp/shell_prop
root@LAPTOP-QJHCANDR:~# ls -l /tmp/shell_prop
-rwsr-xr-x 1 root root 16016 Feb  6 14:38 /tmp/shell_prop
```

On a bien `-rwsr-xr-x 1 root root ... /tmp/shell_prop` comme droits, ce qui confirme que le bit setuid est activé et que le programme est exécutable par tous les utilisateurs.

19. En lançant le programme qui démarre en shell, on peut vérifier que l'uid du shell lancé est bien celui de root (0) avec la commande `id` dans le shell:

```
bluefir  ~  /tmp/shell_prop
root@LAPTOP-QJHCANDR:~# id
```

De plus, si on crée un fichier depuis ce shell, le propriétaire du fichier sera root, car le shell s'exécute avec les privilèges de root grâce au bit setuid:

```
root@LAPTOP-QJHCANDR:~# touch /tmp/test_file
root@LAPTOP-QJHCANDR:~# ls -l /tmp/test_file
-rw-r--r-- 1 root bluefir 0 Feb  6 14:44 /tmp/test_file
```

J'en conclut que les programmes setuid sont utiles car ils permettent à des utilisateurs non privilégiés d'exécuter des programmes avec les privilèges d'un autre utilisateur (souvent root), ce qui est nécessaire pour certaines tâches d'administration de sa machine. Cependant, ils peuvent aussi être dangereux s'ils sont mal configurés ou s'ils contiennent des vulnérabilités, car ils peuvent permettre à un attaquant d'obtenir des privilèges élevés et de compromettre la sécurité du système. On parlerait d'escalade de privilèges, ce qui peut transformer une faille "mineure" sur un compte utilisateur courant en une faille critique donnant accès à tout le système.